Domus IA

Plataforma de diseño arquitectónico asistido por inteligencia artificial

De la descripción del cliente a un modelo 3D estructurado, editable y verificable

Dato	Valor
Documento	Documentación técnica, manuales y trazabilidad del MVP
Versión del producto	0.1.0 (Nivel 1: MVP funcional)
Fecha	17 de octubre de 2026
Contexto	Curso ISII — Semestre 2026-II
Alcance	Prototipo funcional verificable; no es un producto constructivo
Repositorio	domus-ia (monorepo pnpm)

Aviso profesional. Los planos, presupuestos y fichas que genera la plataforma son documentos preliminares. No constituyen un proyecto constructivo ni reemplazan la revisión, el cálculo estructural y la firma de un arquitecto o ingeniero habilitado.

Contenido

1. Resumen ejecutivo
2. Alcance del MVP y criterios de aceptación
3. Requisitos funcionales y no funcionales
4. Casos de uso
5. Arquitectura técnica y decisiones de diseño
6. Modelo de datos
7. Contratos de la API
8. Motor geométrico y capa de inteligencia artificial
9. Seguridad, privacidad y auditoría
10. Plan de pruebas y resultados reales
11. Matriz de trazabilidad
12. Riesgos técnicos y mitigaciones
13. Limitaciones conocidas y trabajo futuro
14. Plan de implementación por fases
15. Manual de instalación y operación
16. Manual de usuario
17. Datasets públicos recomendados y licencias
18. Glosario

1. Resumen ejecutivo

Domus IA es una plataforma web que transforma la descripción de una vivienda —escrita o dictada— en un **modelo arquitectónico tridimensional estructurado**: ambientes con superficie y dimensiones, muros con puertas y ventanas, materiales, mobiliario y entorno. El cliente explora el modelo, lo recorre y lo modifica con instrucciones en lenguaje natural, y la empresa obtiene planos preliminares, una estimación de costos y la ficha del proyecto.

La decisión técnica central es no confundir generación de imágenes con generación arquitectónica. Un modelo de lenguaje no produce geometría válida de forma directa, por lo que la inteligencia artificial **no dibuja**: interpreta la intención del cliente y emite operaciones tipadas (ampliar un ambiente, añadir una ventana, cambiar un material). Un motor paramétrico determinista aplica esas operaciones sobre el modelo existente, verifica restricciones geométricas y funcionales y solo entonces persiste una versión nueva.

El resultado es un MVP funcional, ejecutable y verificable: motor geométrico con 40 pruebas automatizadas, intérprete de lenguaje natural con 22 pruebas, API con 24 verificaciones de extremo a extremo, interfaz 3D, documentos exportables y datos de demostración. Lo que no está probado (realidad virtual y aumentada en hardware real) o no está implementado (interoperabilidad IFC/BIM, dictado por voz) se declara explícitamente en este documento.

Indicador	Valor verificado
Pruebas del motor geométrico	40 aprobadas (4 archivos de prueba)
Pruebas del intérprete de lenguaje natural	22 aprobadas
Verificaciones HTTP de extremo a extremo	24 aprobadas (flujo completo)
Tablas del modelo de datos	20 entidades + proyección geométrica
Endpoints REST documentados	21 (OpenAPI 3.0 en docs/openapi.json)
Operaciones geométricas soportadas	17 tipos
Reglas de validación del modelo	18 códigos de verificación
Compilación de la interfaz	42 módulos, 679 kB (183 kB gzip)

2. Alcance del MVP y criterios de aceptación

El alcance del Nivel 1 se definió para demostrar el ciclo completo de valor: describir, generar, explorar, modificar, comparar y documentar. Todo lo demás se difiere de forma consciente para no construir una vitrina visual incapaz de editar modelos de forma fiable.

2.1 Criterios de aceptación de la primera versión

Criterio	Estado	Evidencia / observación
Registrar un proyecto de vivienda	Cumplido	POST /api/projects con terreno, programa y presupuesto
Introducir una descripción de la vivienda	Cumplido	Panel de instrucciones + POST /instructions

Criterio	Estado	Evidencia / observación
Transformar la descripción en requisitos estructurados	Cumplido	ParsedIntent con estilo, niveles, áreas, features
Generar un modelo 3D básico desde los requisitos	Cumplido	Motor paramétrico determinista (40 pruebas)
Explorar el modelo con controles interactivos	Cumplido	Visor Three.js: órbita, zoom, selección, capas
Solicitar una modificación en lenguaje natural	Cumplido	22 pruebas de interpretación
Actualizar sin perder componentes no afectados	Cumplido	Operaciones locales + diff por elemento
Guardar y recuperar el proyecto	Cumplido	Versiones inmutables en PostgreSQL
Comparar al menos dos versiones	Cumplido	GET /compare/:a/:b + diálogo de comparación
Consultar dimensiones de los componentes	Cumplido	Panel de propiedades + export JSON
Exportar documentación preliminar	Cumplido	Planta SVG, ficha PDF, export JSON
Informar limitaciones y validaciones pendientes	Cumplido	Campo warnings, panel de estado, avisos en documentos
VR/AR probadas en dispositivos compatibles	Pendiente	Implementado con WebXR; requiere hardware y navegador compatibles

Los criterios marcados como pendientes no son omisiones silenciosas: se identifican en la interfaz, en la API y en la sección 13 de este documento.

3. Requisitos funcionales y no funcionales

Los requisitos se derivaron del encargo y se agruparon por módulo. Cada uno indica su estado real en el MVP: **operativo**, **parcial** (existe una versión limitada y documentada) o **no implementado**.

3.1 Requisitos funcionales

ID	Requisito	Módulo	Estado
RF-01	Registrar clientes con datos de contacto	1	Operativo
RF-02	Crear proyectos de diseño de viviendas	1	Operativo
RF-03	Registrar dimensiones y características del terreno	1	Operativo
RF-04	Definir presupuesto, número de pisos y estilo arquitectónico	1	Operativo
RF-05	Guardar los avances del proyecto de forma automática	1	Operativo

ID	Requisito	Módulo	Estado
RF-06	Consultar proyectos anteriores	1	Operativo
RF-07	Duplicar proyectos para explorar alternativas	1	No implementado
RF-08	Gestionar el estado del proyecto (borrador… finalizado)	1	Operativo
RF-09	Interpretar la descripción del cliente y detectar ambigüedades	2	Operativo
RF-10	Generar una propuesta preliminar de distribución	2/3	Operativo
RF-11	Construir un modelo 3D estructurado y editable	3	Operativo
RF-12	Visualizar y navegar el modelo con controles interactivos	3	Operativo
RF-13	Modificar el diseño mediante lenguaje natural	6	Operativo (reglas)
RF-14	Conservar los componentes no afectados al modificar	6	Operativo
RF-15	Deshacer, rehacer y recuperar versiones anteriores	6	Operativo
RF-16	Comparar dos versiones del diseño	6	Operativo
RF-17	Personalizar materiales, acabados y mobiliario	7	Operativo
RF-18	Actualizar la estimación de costos al cambiar materiales	7	Operativo
RF-19	Generar planos preliminares y documentación	8	Operativo
RF-20	Exportar documentos (SVG, PDF, JSON)	8	Operativo
RF-21	Panel de control con indicadores del negocio	9	Operativo
RF-22	Autenticación, autorización y perfiles de usuario	10	Operativo
RF-23	Auditoría y registro de eventos relevantes	10	Operativo
RF-24	Recorrido en realidad virtual y aumentada	4/5	Parcial (sin hardware)
RF-25	Catálogo de mobiliario y materiales con precios	7	Operativo (demo)

ID	Requisito	Módulo	Estado
RF-26	Revisión y aprobación profesional de versiones	8	Operativo
RF-27	Dictado por voz de las instrucciones	2/6	No implementado
RF-28	Importación y exportación BIM (IFC)	8	No implementado

3.2 Requisitos no funcionales

ID	Categoría	Requisito y cumplimiento
RNF-01	Usabilidad	Interfaz en español, adaptable a escritorio y tableta; mensajes de error explicativos y ayudas contextuales (chips de instrucciones, panel de propiedades).
RNF-02	Rendimiento	Generación del modelo completa en menos de 300 ms para el escenario de demostración; geometría servida como JSON y render local en el navegador; actualización localizada por operación.
RNF-03	Confiabilidad	Cada operación se aplica sobre una copia del modelo, se valida y solo entonces se consolida; historial de versiones inmutable y restauración disponible.
RNF-04	Seguridad	JWT firmado, contraseñas con bcrypt (10 rondas), RBAC por rol, validación de DTO, auditoría de eventos y protección de los endpoints.
RNF-05	Interoperabilidad	API REST documentada con OpenAPI 3.0; exportación JSON, SVG y PDF. IFC/BIM previsto para la siguiente fase.
RNF-06	Escalabilidad	Arquitectura modular: API sin estado, motor puro y catálogo ampliable; PostgreSQL en producción y PGlite para desarrollo sin servicios.
RNF-07	Mantenibilidad	Monorepo TypeScript estricto, paquetes con responsabilidad única y 62 pruebas automatizadas como red de seguridad.
RNF-08	Portabilidad	Variables de entorno documentadas, migraciones versionadas y despliegue con Docker Compose.
RNF-09	Privacidad	Datos personales mínimos; no se registran credenciales ni contenido sensible en los logs de auditoría.

ID	Categoría	Requisito y cumplimiento
RNF-10	Trazabilidad	Cada cambio queda en change_history con su diff, autor y versión; los eventos de negocio en audit_events.
RNF-11	Accesibilidad	Contraste suficiente, etiquetas en formularios y navegación por teclado en los controles principales (revisión manual, sin auditoría WCAG formal).
RNF-12	Documentación	Documento técnico, manual de usuario e instalación, OpenAPI exportado y matriz de trazabilidad requisitos → pruebas.

4. Casos de uso

Se documentan los diez casos de uso que sostienen el MVP. Los flujos describen el comportamiento implementado, no el deseado.

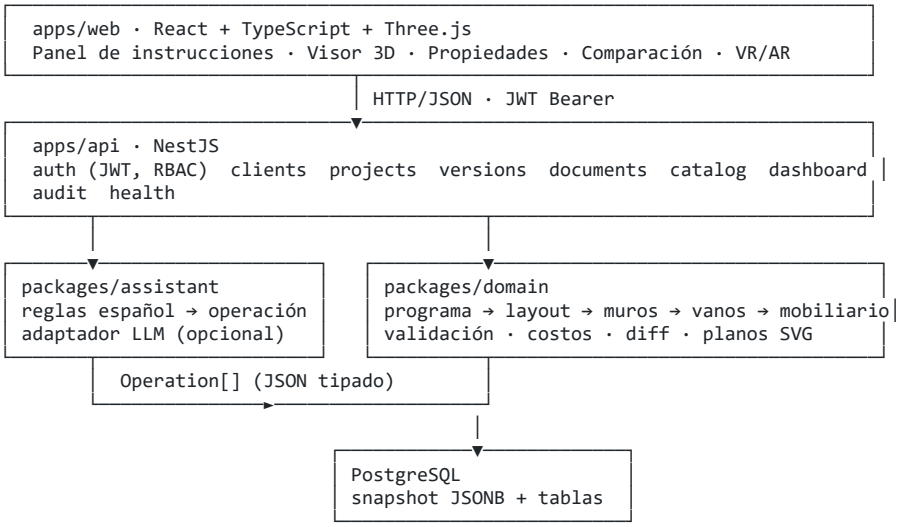
CU	Actor	Objetivo y flujo principal	Criterio de aceptación
CU-01	Asesor / Cliente	Registrar un proyecto: datos del cliente, terreno, programa (dormitorios, baños, cochera), estilo y presupuesto.	El proyecto se crea con la versión 1 generada y estado «en diseño».
CU-02	Cliente	Describir la vivienda en lenguaje natural; el sistema extrae requisitos y pregunta lo que falta.	Se obtiene un ParsedIntent con requisitos y, si falta información, preguntas concretas.
CU-03	Cliente	Modificar un ambiente («amplía la cocina a 16 m²»).	El modelo se actualiza, el resto de ambientes se conserva y se crea la versión n+1.
CU-04	Cliente	Comparar la versión vigente con la anterior.	Se muestran áreas, costos y el listado de cambios por elemento.
CU-05	Asesor	Descargar plano, ficha y exportación del modelo.	SVG y PDF se generan desde la geometría; el JSON incluye modelo, costos y validaciones.
CU-06	Arquitecto	Revisar una versión y aprobarla o devolverla.	Se registra la aprobación y el proyecto cambia de estado.
CU-07	Cliente	Deshacer un cambio.	Se restaura la versión anterior como versión nueva, sin perder el historial.
CU-08	Administrador	Consultar indicadores del negocio.	El panel muestra datos reales calculados en SQL sobre los proyectos y versiones.
CU-09	Administrador / Arquitecto	Consultar materiales y mobiliario disponibles.	Catálogo con precios, proveedor y disponibilidad (datos de demostración).

CU	Actor	Objetivo y flujo principal	Criterio de aceptación
CU-10	Cliente con visor	Recorrer la vivienda en VR/AR.	Si el dispositivo lo soporta se inicia la sesión WebXR; si no, se informa y se mantiene el modo 3D.

5. Arquitectura técnica y decisiones de diseño

La arquitectura es modular y separa de forma explícita la interfaz, la lógica de negocio y el motor geométrico, tal como exige el encargo. La regla de dependencia es unidireccional: la web habla con la API, la API orquesta el intérprete y el motor, y el motor no conoce ni la red ni la base de datos.

5.1 Vista de componentes



5.2 Responsabilidades por paquete

Paquete / carpeta	Responsabilidad	Dependencias
packages/domain	Modelo arquitectónico, generador paramétrico, operaciones, validaciones, costos, diff y planos SVG.	Ninguna (TypeScript puro)
packages/assistant	Interpretación de instrucciones en español a operaciones; adaptador opcional a LLM con validación de contrato.	@domus/domain (tipos)
apps/api	Autenticación, RBAC, proyectos, versiones, documentos, panel, auditoría y persistencia.	NestJS, pg/PGlite, pdfkit, domain, assistant
apps/web	Interfaz de usuario, visor 3D, comparación y descargas.	React, Three.js, Vite
apps/api/migrations	Esquema SQL versionado (PostgreSQL).	—

5.3 Decisiones de arquitectura (ADR)

ADR	Decisión	Justificación y consecuencia
ADR-01	Modelo estructurado en lugar de malla o imagen generada por IA.	Permite edición por elemento, validación y documentación. Una imagen no es editable ni verificable; el modelo sí. Consecuencia: se renuncia a la espectacularidad de una imagen generada.
ADR-02	Motor paramétrico determinista propio (treemap squarified + optimización de adyacencias + bloque reservado para escalera).	Resultados reproducibles, auditables y probables con pruebas unitarias. Consecuencia: la distribución es conceptual y requiere revisión profesional.
ADR-03	Contrato explícito de operaciones tipadas entre IA y geometría.	La IA nunca toca la geometría: emite JSON validado. Consecuencia: se pueden incorporar nuevos modelos de lenguaje sin cambiar el motor.
ADR-04	Validación transaccional por operación con rollback lógico.	Una instrucción que rompe la planta se rechaza sin afectar las anteriores. Consecuencia: el usuario recibe explicaciones en lugar de un modelo corrupto.
ADR-05	Snapshot JSONB por versión + proyección relacional.	Versionado inmutable y consultas SQL para indicadores. Consecuencia: la proyección debe mantenerse sincronizada al crear una versión.
ADR-06	PGlite (PostgreSQL embebido) en desarrollo y PostgreSQL en producción.	El MVP se ejecuta sin servicios externos y el SQL es el mismo. Consecuencia: PGlite es monousuario; en producción se usa PostgreSQL con pool de conexiones.
ADR-07	Intérprete por reglas como camino principal y LLM como adaptador opcional.	Funciona sin credenciales, sin costo y sin latencia; el LLM mejora la cobertura cuando está disponible y su salida se valida antes de usarse.
ADR-08	La interfaz no importa el paquete de dominio.	Evita acoplar el cliente al motor y problemas de interoperabilidad de módulos; el contrato se refleja en tipos del frontend. Consecuencia: hay que mantener ambos tipos sincronizados.

5.4 Tecnologías

Componente	Tecnología	Función
Interfaz web	React 18 + TypeScript + Vite	Gestión de proyectos e interacción
Experiencia 3D	Three.js (WebGL)	Render, navegación, selección y capas
Realidad virtual / aumentada	WebXR	Sesiones inmersivas cuando el dispositivo las soporta
Motor geométrico	TypeScript propio	Generación y modificación de componentes
Backend	Node.js 20+ + NestJS 10	API, reglas de negocio y orquestación

Componente	Tecnología	Función
Base de datos	PostgreSQL 16 (PGlite en local)	Proyectos, versiones, catálogo y auditoría
Inteligencia artificial	API compatible con chat completions (opcional)	Interpretación de instrucciones; validada por reglas
Documentos	SVG propio + PDFKit	Planos preliminares y ficha del proyecto
Despliegue	Docker Compose + Nginx	API, base de datos e interfaz

6. Modelo de datos

El esquema relacional cubre las veinte entidades exigidas por el encargo. La geometría se conserva como **snapshot JSONB inmutable** por versión (`design_versions.model`) y se proyecta a tablas relacionales para consultas e indicadores. Esa doble representación evita migraciones geométricas costosas y permite auditar cada versión tal como se entregó.

Entidad	Propósito	Relaciones principales
roles / permissions / role_permissions	RBAC: perfiles y permisos	users → roles; roles ↔ permissions
users	Cuentas de acceso	1:N clients, projects.owner_id, approvals.reviewer_id
clients	Datos de contacto del cliente	1:N projects
projects	Proyecto de diseño y su estado	N:1 clients; 1:1 terrains; 1:N design_versions
terrains	Dimensiones, retiros y orientación	1:1 projects
design_versions	Snapshot del modelo, costo y validación	N:1 projects; auto-referencia parent_version
buildings / levels / rooms	Proyección analítica de la geometría	buildings → levels → rooms
elements	Muros y mobiliario proyectados	N:1 levels (kind: wall | furniture)
materials / furniture_catalog	Catálogo de acabados y mobiliario	N:1 suppliers
suppliers	Proveedores del catálogo	1:N materials
modification_instructions	Instrucciones recibidas y operaciones	N:1 projects
change_history	Diff entre versiones	N:1 projects
documents	Metadatos de documentos generados	N:1 projects
approvals	Revisiones y aprobaciones profesionales	N:1 projects; N:1 users
audit_events	Eventos de auditoría	N:1 users
schema_migrations	Control de migraciones aplicadas	—

6.1 Reglas e integridad

- **Claves primarias UUID** generadas en la aplicación (texto), lo que permite migrar entre motores sin extensiones adicionales.
- **Claves foráneas con borrado en cascada** en la jerarquía proyecto → versiones → proyección, para evitar registros huérfanos.

- **Restricción de estado** (`CHECK`) en `projects.status` y en `approvals.status`.
- **Unicidad** de `(project_id, version)` y de `lower(email)` en usuarios.
- **Índices** en `projects(client_id, status)`, `design_versions(project_id, version DESC)`, `rooms(level_id)`, `audit_events(created_at DESC)` y `change_history(project_id)`.
- **Importes** en `NUMERIC(14,2)` y superficies en `NUMERIC(10,2)`; el dinero nunca se almacena en coma flotante.

6.2 Recuperación de versiones

Cada modificación crea una fila nueva en `design_versions` con `parent_version` apuntando a la anterior. Restaurar una versión no la sobrescribe: copia su snapshot como una versión nueva, de modo que el historial completo permanece íntegro y auditable (RF-15).

7. Contratos de la API

La API es REST sobre JSON, con autenticación Bearer y validación de entrada con DTO. La especificación completa se genera automáticamente y se exporta a `docs/openapi.json`; la documentación interactiva está disponible en `/api/docs`.

Método y ruta	Descripción	Roles
GET /api/health	Estado del servicio, base de datos y modelo de lenguaje	Público
POST /api/auth/register	Registro de cuenta y emisión de token	Público
POST /api/auth/login	Inicio de sesión	Público
GET /api/auth/me	Perfil autenticado	Todos
GET /api/clients · POST /api/clients	Listado y alta de clientes	Admin, arquitecto, asesor
GET /api/projects	Listado de proyectos (filtro por estado)	Todos
POST /api/projects	Crea proyecto + terreno + versión 1 generada	Todos
GET /api/projects/:id	Detalle con versión vigente, historial e instrucciones	Todos
PATCH /api/projects/:id/status	Cambia el estado del proyecto	Admin, arquitecto, asesor
POST /api/projects/:id/generate	Nueva propuesta desde los requisitos vigentes	Todos
POST /api/projects/:id/instructions	Interpreta y aplica una instrucción	Todos
POST /api/projects/:id/restore	Restaura una versión como versión nueva	Todos
GET /api/projects/:id/versions	Lista de versiones	Todos
GET /api/projects/:id/versions/:v	Versión con modelo, costo y validación	Todos
GET /api/projects/:id/compare/:a/:b	Diferencias entre dos versiones	Todos
GET /api/projects/:id/changes	Historial de cambios	Todos
POST /api/projects/:id/approvals	Registra revisión profesional	Admin, arquitecto
GET /api/projects/:id/approvals	Revisiones del proyecto	Todos
GET /api/projects/:id/plan.svg	Planta preliminar en SVG (por nivel)	Todos

Método y ruta	Descripción	Roles
GET /api/projects/:id/report.pdf	Ficha del proyecto en PDF	Todos
GET /api/projects/:id/export.json	Exportación completa (modelo, costos, validación)	Todos
GET /api/catalog/materials · furniture · suppliers · styles	Catálogo de referencia	Todos
GET /api/dashboard/summary	Indicadores calculados en SQL	Todos
GET /api/audit	Últimos eventos de auditoría	Admin

Contrato de la modificación. POST /projects/:id/instructions devuelve la versión creada, el modelo resultante, las operaciones aplicadas y rechazadas, los avisos del motor, el diff y el costo actualizado. Si ninguna operación puede aplicarse responde 422 con el motivo y las preguntas pendientes.

8. Motor geométrico y capa de inteligencia artificial

8.1 Del texto a la geometría

```
texto del cliente
   │  normalización (tildes, números en palabras, unidades)
   ▼
ParsedIntent ── requisitos estructurados (estilo, niveles, dormitorios, terreno, presupuesto)
   │        └── preguntas cuando falta información
   ▼
Operation[]  (JSON tipado y validado)
   │
   ▼
applyOperations() ── por operación: copia → aplicar → validar → consolidar o rechazar
   │
   ▼
DesignModel ── snapshot JSONB + proyección SQL + diff + costo + plano SVG
   │
   ▼
Visor 3D / documentos / VR-AR
```

8.2 Generación de la propuesta inicial

1. **Programa arquitectónico**: se calculan ambientes, superficies y nivel a partir del estilo, el presupuesto y el programa familiar, con mínimos por tipología.
2. **Huella edificable**: terreno menos retiros; si el programa no cabe se reduce de forma proporcional y se emiten avisos.
3. **Distribución**: reparto por bandas (treemap squarified normalizado), optimización de adyacencias funcionales y equilibrio de superficies respecto del programa.
4. **Circulación vertical**: la escalera se reserva como bloque fijo y se reutiliza en las plantas superiores para garantizar continuidad.
5. **Muros y vanos**: se derivan los muros exteriores e interiores, la puerta de acceso, la puerta de cochera, las puertas interiores y las ventanas según el estilo.
6. **Mobiliario**: amoblamiento determinista por tipología, recortado al interior del ambiente.
7. **Validación**: geometría, accesibilidad, iluminación natural, alturas, ocupación y presupuesto.
8. **Costos**: estimación paramétrica por superficies, vanos y mobiliario, con contingencia y honorarios.

8.3 Operaciones de modificación

Operación	Parámetros	Efecto
resize_room	roomId, targetArea	Mueve tabiques hacia los vecinos; conserva el área total de la planta
add_room	type, level, targetArea	Divide un ambiente existente y crea el nuevo con su puerta
remove_room	roomId	Reparte la superficie entre los vecinos que cubren un lado completo
add_window	roomId, width, height, sill	Abre un vano en el muro exterior del ambiente
windows_floor_to_ceiling	scope, roomId/level	Convierte ventanas en ventanales de piso a techo

Operación	Parámetros	Efecto
add_door / move_door	roomId / doorId, delta	Abre o desplaza una puerta dentro de su muro
set_room_floor / set_room_color	roomId, materialId / color	Cambia acabado o color de un ambiente
set_facade_color / set_facade_material	color / materialId	Actualiza la fachada completa
set_roof_material	materialId	Cambia la cubierta
set_style	style	Aplica la paleta y los acabados del estilo
add_furniture / remove_furniture	roomId, type	Incorpora o retira mobiliario
add_level	—	Duplica la planta superior como base editable
reduce_budget	percent	Selecciona materiales económicos y retira elementos premium

8.4 Reglas de validación

Toda operación se valida antes de persistirse. Las reglas se agrupan en errores (bloquean el cambio) y advertencias (se aceptan y se informan).

Código	Severidad	Verificación
ROOM_OVERLAP	Error	Dos ambientes no pueden superponerse (tolerancia 3 cm por redondeo)
ROOM_OUT_OF_FOOTPRINT	Error	El ambiente debe quedar dentro de la huella
ROOM_UNUSABLE	Error	Superficie inferior a 1,5 m²
ROOM_UNREACHABLE	Error	Todo ambiente debe ser accesible desde el hall
OPENING_INVALID	Error	El vano no puede exceder su muro ni su altura
WALL_DEGENERATE / WALL_OUT_OF_FOOTPRINT	Error	Muros con longitud nula o fuera de la huella
FOOTPRINT_OUT_OF_SETBACK	Error	La huella no puede invadir los retiros
SETBACK_INVALID / TERRAIN_INVALID	Error	Retiros o terreno inconsistentes
NO_LEVELS / NO_STAIRS / NO_STAIRS_LEVEL	Error	Faltan plantas o circulación vertical
ROOM_TOO_SMALL	Advertencia	Por debajo del mínimo recomendado por tipología
ROOM_NO_DAYLIGHT	Advertencia	Ambiente sin ventana en fachada
COVERAGE_EXCEEDED	Advertencia	Ocupación del suelo superior al 75 %
HEIGHT_EXCEEDED	Advertencia	Altura total superior a 9,5 m
STAIRS_MISALIGNED	Advertencia	Escalera no alineada entre plantas
NO_ENTRANCE	Advertencia	No se detectó puerta de acceso
BUDGET_EXCEEDED	Advertencia	Estimación por encima del presupuesto declarado

8.5 Capa de inteligencia artificial

El intérprete determinista reconoce verbos de acción, tipologías de ambiente, colores, materiales, estilos y cantidades en español, y separa instrucciones compuestas («amplía la sala y reduce el comedor»). Cuando existe un proveedor de lenguaje configurado, se le envía el estado del modelo y el catálogo de operaciones; la respuesta se valida contra el contrato y, si no lo cumple, se descarta y se usan las reglas. Así se evita que una alucinación llegue a la geometría.

- La IA **no** genera geometría ni imágenes: genera operaciones explícitas y verificables (ADR-03).
- Las operaciones desconocidas devueltas por el modelo se descartan antes de aplicarse.
- Toda respuesta incluye explicación, confianza y notas de interpretación.
- Si falta información, el asistente formula preguntas concretas antes de proceder.

9. Seguridad, privacidad y auditoría

Control	Implementación	Alcance
Autenticación	JWT firmado (HS256) con expiración configurable	Todas las rutas salvo /health, login y registro
Contraseñas	bcrypt con 10 rondas de sal	Nunca se devuelven ni se registran
Autorización	RBAC por rol con guard global y decorador @Roles	admin, architect, advisor, client
Validación de entrada	class-validator + ValidationPipe (whitelist)	Todos los DTO
Auditoría	audit_events con actor, acción, entidad, payload e IP	Altas, cambios de estado, instrucciones y aprobaciones
Trazabilidad del diseño	change_history con diff y versión de origen	Cada modificación
Protección de API	Tokens Bearer; CORS restringido al origen web	Todas las rutas
Secretos	.env fuera del control de versiones; .env.example sin credenciales	Repositorio

9.1 Matriz de permisos

Rol	Proyectos	Aprobar	Catálogo	Auditoría	Usuarios
Administrador	Leer y escribir	Sí	Leer y escribir	Sí	Sí (previsto)
Arquitecto	Leer y escribir	Sí	Leer	No	No
Asesor comercial	Leer y escribir	No	Leer	No	No
Cliente	Leer y escribir (sus proyectos)	No	Leer	No	No

9.2 Riesgos de seguridad y brechas conocidas

- No se implementaron refresh tokens ni revocación de sesión: el token vive hasta su expiración.
- No hay autenticación multifactor ni política de contraseñas más allá de la longitud mínima.
- El aislamiento por propietario está previsto en el modelo (owner_id, client_id) pero la API no filtra aún proyectos por cliente; debe añadirse antes de un uso multiusuario real.
- Los documentos se generan en memoria; el almacenamiento de objetos (S3/MinIO) queda pendiente.
- No se ejecutó una prueba de penetración ni un escaneo de dependencias en este entorno.

10. Plan de pruebas y resultados reales

Las pruebas se ejecutaron en el entorno de desarrollo del proyecto. Los resultados que se muestran son los obtenidos, no estimaciones.

Suite	Alcance	Resultado	Comando
Motor geométrico	Generación, distribución, muros, mobiliario, operaciones, validación, costos, diff y SVG	40/40	pnpm --filter @domus/domain test
Intérprete NL	Generación, 15 tipos de modificación, compuestas, ambigüedad y adaptador LLM	22/22	pnpm --filter @domus/assistant test
Extremo a extremo	Autenticación, proyecto, instrucción, parcial, comparación, documentos, panel, auditoría, OpenAPI	24/24	pnpm build && pnpm e2e:api
Compilación de la interfaz	TypeScript estricto + build de producción	Correcta	pnpm --filter @domus/web build
Migraciones y datos demo	Esquema completo + 3 proyectos con versiones	Correcta	pnpm --filter @domus/api seed

10.1 Casos cubiertos por la suite del motor

- El modelo generado pasa todas las validaciones y es determinista entre ejecuciones.
- Se respetan plantas, dormitorios, baños, cochera y terraza solicitados.
- Los ambientes cubren la huella sin solaparse y respetan retiros y ocupación.
- Todos los ambientes son accesibles y la escalera queda alineada entre plantas.
- La ampliación de la cocina conserva el resto de ambientes y el área total de la planta.
- Una operación inválida no afecta a las operaciones válidas de la misma instrucción.
- Agregar y eliminar ambientes mantiene la teselación de la planta.
- Convertir ventanas en ventanales, cambiar estilos y reducir presupuesto actualizan el modelo.
- El diff identifica cambios estructurales frente a cambios de acabado.
- La planta SVG incluye ambientes, superficies y cotas; el PDF y el JSON se generan completos.

10.2 Estrategia de pruebas

Nivel	Enfoque	Herramienta
Unitario / integración de dominio	Funciones puras con aserciones sobre geometría, áreas, validaciones y costos	Vitest
Integración de NLU	Frases reales del encargo contra las operaciones esperadas	Vitest + mock de fetch
Extremo a extremo	API levantada en proceso con base temporal y llamadas HTTP reales	Node fetch + NestFactory
Regresión visual / render	Pendiente: captura del visor en navegador automatizado	No implementado
Carga	Pendiente: no se midió concurrencia ni latencia bajo carga	No implementado

11. Matriz de trazabilidad

Cada requisito funcional se vincula con su implementación y con la prueba que lo respalda. La misma matriz se exporta a `docs/trazabilidad.csv` para su revisión en hoja de cálculo.

Requisito	Implementación	Prueba o evidencia
RF-01 · Registrar clientes	apps/api/clients	E2E: alta de proyecto con cliente
RF-02 · Crear proyectos	apps/api/projects	E2E: «proyecto creado con versión 1»
RF-03 · Registrar terreno	apps/api/projects + terrain	E2E: creación con terreno 10×20
RF-04 · Presupuesto, pisos y estilo	CreateProjectDto + program	Domain: «respeta el número de plantas, dormitorios y baños»
RF-05 · Guardado automático	Persistencia por versión	E2E: historial y versiones listadas
RF-06 · Consultar proyectos	GET /projects	E2E: listado y detalle
RF-07 · Duplicar proyectos	—	No implementado (ver sección 13)
RF-08 · Estados del proyecto	PATCH /projects/:id/status	E2E: aprobación cambia el estado
RF-09 · Interpretar descripción y ambigüedades	packages/assistant	NLU: «extrae requisitos» y «detecta información faltante»
RF-10 · Propuesta de distribución	program + layout	Domain: «distribuye los ambientes sin solaparse»
RF-11 · Modelo 3D estructurado	generateDesign + deriveWalls	Domain: «genera muros con vanos y acceso»
RF-12 · Visualizar y navegar	HouseScene (Three.js)	Web: build + verificación manual del visor
RF-13 · Modificar por lenguaje natural	instruct + applyOperations	NLU: 15 casos; E2E: instrucción y versión nueva
RF-14 · Conservar lo no afectado	resize con reflujo local	Domain: «amplía la cocina conservando el resto»
RF-15 · Deshacer/rehacer/versiones	restore + design_versions	E2E: restauración y versiones
RF-16 · Comparar versiones	diffModels + /compare	Domain: diff; E2E: comparación 1↔n
RF-17 · Personalizar materiales y mobiliario	set_* + add_furniture	NLU: porcelanato, isla, colores
RF-18 · Actualizar costos	estimateCost en cada versión	Domain: costos; E2E: costo por versión
RF-19 · Planos y documentación	renderPlanSvg + reportPdf	Domain: SVG; E2E: SVG y PDF
RF-20 · Exportar documentos	export.json + SVG/PDF	E2E: exportación completa
RF-21 · Panel de control	GET /dashboard/summary	E2E: indicadores
RF-22 · Usuarios y seguridad	auth + guards	E2E: registro, perfil y acceso denegado
RF-23 · Auditoría	audit_events	E2E: auditoría con eventos
RF-24 · VR/AR	HouseScene.enterVR/enterAR	No probado en dispositivo (limitación)
RF-25 · Catálogo de materiales y mobiliario	materials + furniture_catalog	E2E: creación con catálogo sembrado
RF-26 · Revisión y aprobación	POST /approvals	E2E: «aprobación registrada»
RF-27 · Dictado por voz	—	No implementado (ver sección 13)
RF-28 · Interoperabilidad IFC/BIM	—	No implementado (ver sección 13)

12. Riesgos técnicos y mitigaciones

Riesgo	Impacto	Mitigación aplicada o prevista
Interpretar «generar una casa» como generar una imagen	Alto	Arquitectura y contrato de operaciones; el modelo es la única fuente de verdad
La IA produce operaciones inválidas o alucinadas	Alto	Validación de contrato + validación geométrica + rechazo transaccional
La edición rompe la distribución (huecos, solapes)	Alto	Teselación validada, reflujo local y rechazo con explicación
Catálogo de precios desactualizado o ficticio	Medio	Marcado como demostración; sustitución por datos de proveedores con vigencia
Confundir documento preliminar con proyecto constructivo	Alto	Aviso explícito en la interfaz, en la API y en cada PDF
Rendimiento del visor con muchos ambientes	Medio	Geometría simple, instancing previsto, sin sombras dinámicas
Dependencia de un proveedor de IA	Medio	Adaptador opcional y modo reglas sin credenciales
Sesión sin revocación	Medio	Expiración corta y registro de eventos; refresh tokens en la siguiente fase
Ausencia de pruebas de carga y de seguridad ofensiva	Medio	Declarado como pendiente antes de producción
VR/AR no verificadas en hardware	Bajo/Medio	Implementación WebXR con degradación explícita a 3D

13. Limitaciones conocidas y trabajo futuro

Esta sección es deliberadamente explícita: el encargo prohíbe presentar módulos simulados como completamente implementados.

13.1 Limitaciones funcionales

Limitación	Comportamiento actual	Trabajo futuro
Ampliación de un ambiente sin holgura en los vecinos	Se aplica solo la parte posible y se devuelve una advertencia explicativa	Layout por árbol de particiones (slicing) para reflujo exacto
Alineación de la escalera entre plantas	Se reserva un bloque fijo; se advierte si no puede alinearse	Núcleo de circulación con escalera y ductos reservados por diseño
Realidad virtual y aumentada	Código WebXR con degradación a 3D; sin probar en visor	Pruebas en Meta Quest/ARKit y guía de compatibilidad
Interoperabilidad BIM/IFC	No implementada; exportación JSON y documentos	Importador/exportador IFC 4.x y mapeo a elementos
Dictado por voz	No implementado (solo texto)	Web Speech API con confirmación previa
Duplicar proyectos	No implementado	Clonado de proyecto y versión base
Presupuesto	Estimación paramétrica con catálogo de demostración	Cotización con proveedores, IGV y análisis de precios unitarios

Limitación	Comportamiento actual	Trabajo futuro
Planos	Planta por nivel en SVG con áreas y cotas generales	Cortes, elevaciones, instalaciones y cajetín normado
Colaboración multiusuario en tiempo real	No implementada	Presencia, comentarios y bloqueo optimista por versión

13.2 Limitaciones técnicas

- PGlite es monousuario: no se validó concurrencia real; para producción se usa PostgreSQL con pool de conexiones.
- La API no filtra proyectos por propietario; el RBAC limita roles, no la pertenencia de cada registro.
- No se ejecutaron pruebas de carga, de penetración ni auditoría WCAG formal.
- El `docker-compose.yml` y los Dockerfile no pudieron ejecutarse en el entorno de desarrollo (sin Docker); se validaron de forma estática.
- El modelo admite hasta tres plantas y ambientes rectangulares; no representa formas curvas ni desniveles.

Regla de honestidad del proyecto. Todo lo que no se ejecutó se reporta como no verificado; lo que no se implementó se reporta como pendiente. Las cifras de pruebas corresponden a ejecuciones reales registradas en este entorno.

14. Plan de implementación por fases

Fase	Contenido	Estado
Fase 1 · Análisis y especificación	Actores, casos de uso, requisitos, criterios de aceptación	Completada
Fase 2 · Diseño técnico	Arquitectura, modelo de datos, contratos, motor y formato interno	Completada
Fase 3 · Prototipo funcional	Autenticación, proyectos, requisitos, vivienda paramétrica, visor y persistencia	Completada
Fase 4 · Integración de IA	Asistente, interpretación, operaciones estructuradas y validaciones	Completada (modo reglas)
Fase 5 · VR y AR	Recorridos inmersivos y visualización aumentada	Implementada, sin validación en hardware
Fase 6 · Documentación y presupuestos	Planos, catálogo, estimaciones y exportaciones	Completada (preliminar)
Fase 7 · Seguridad y pruebas	Pruebas unitarias, integración, validación geométrica	Parcial: falta carga y seguridad ofensiva
Fase 8 · Despliegue	Docker, manuales, datos demo y limitaciones	Completada (Compose no ejecutado)

14.1 Siguientes pasos recomendados

9. Sustituir el reparto por un árbol de particiones para que la edición sea exacta en todos los casos.

10. Añadir aislamiento por propietario, refresh tokens y revocación de sesión.
11. Validar VR/AR en dispositivos reales y documentar la matriz de compatibilidad.
12. Integrar IFC para lectura y escritura de modelos BIM.
13. Conectar el catálogo a precios reales de proveedores con fecha de vigencia.
14. Automatizar pruebas de render (captura del visor) y de carga.

15. Manual de instalación y operación

15.1 Requisitos

- Node.js 20 o superior y pnpm 9 o superior.
- Opcional: Docker y Docker Compose para el despliegue con PostgreSQL y Nginx.
- Opcional: una clave de un proveedor compatible con la API de chat completions para el modo LLM.

15.2 Instalación local (sin servicios externos)

```
pnpm install
pnpm build
pnpm --filter @domus/api migrate
pnpm --filter @domus/api seed
pnpm dev:api
```

La API queda en http://localhost:3000/api, la documentación en /api/docs y la interfaz compilada se sirve en la raíz. Para desarrollo de la interfaz con recarga en caliente use `pnpm dev:web` (http://localhost:5173, con proxy hacia /api).

15.3 Variables de entorno

Variable	Uso	Valor por defecto
PORT	Puerto de la API	3000
DB_DRIVER	pglite o postgres	pglite
DATABASE_URL	Cadena de conexión PostgreSQL	postgres:// domus:domus@localhost:5432/domus
PGLITE_DIR	Directorio de datos embebidos	data/pglite
JWT_SECRET	Firma de tokens (cambiar en producción)	—
JWT_EXPIRES_IN	Vigencia del token	8h
LLM_API_KEY / LLM_BASE_URL / LLM_MODEL	Proveedor de lenguaje opcional	sin configurar
WEB_ORIGIN	Origen permitido por CORS	http://localhost:5173

15.4 Despliegue con Docker

```
cp .env.example .env
docker compose up --build
docker compose exec api node apps/api/dist/scripts/migrate.js
docker compose exec api node apps/api/dist/scripts/seed.js
```

Interfaz en http://localhost:8080 y API en http://localhost:3000/api. El servicio web usa Nginx y redirige /api hacia el contenedor de la API.

15.5 Operación y diagnóstico

- `GET /api/health` informa el estado del servicio, del motor de base de datos y del proveedor de lenguaje.

- Las migraciones son idempotentes y se registran en `schema_migrations`; se ejecutan también al arrancar la API.
- El seed no duplica datos: si ya existen usuarios, no vuelve a sembrar.
- Los errores de validación devuelven 400 con detalle; las instrucciones imposibles, 422 con explicación.

16. Manual de usuario

16.1 Ingresar

Abra la aplicación e ingrese con una cuenta de demostración (clave `Domus2026!`): `admin@domus.local`, `arquitecto@domus.local`, `asesor@domus.local` o `cliente@domus.local`.

16.2 Crear un proyecto

15. Pulse «Nuevo proyecto» y complete nombre, estilo, número de plantas, dormitorios, baños, cochera, personas y presupuesto.
16. Indique las dimensiones del terreno en metros (ancho y fondo).
17. Al confirmar, el motor genera la primera propuesta y abre el diseñador.

16.3 Explorar la vivienda

- Arrastre con el botón izquierdo para orbitar y use la rueda para acercar o alejar.
- Cambie de planta con las pestañas de la barra superior o use «Ver todo».
- Oculte muros, cubierta o mobiliario para inspeccionar el interior.
- Haga clic en un ambiente para seleccionarlo: el panel derecho muestra superficie, dimensiones, altura, piso, color y número de puertas y ventanas.

16.4 Modificar con lenguaje natural

Escriba la instrucción en el panel izquierdo y envíela. Ejemplos que el sistema entiende:

- «Amplía la cocina a 16 m²».
- «Agrega una isla central a la cocina».
- «Cambia la fachada a tonos blancos y grises».
- «Cambia las ventanas por ventanales de piso a techo».
- «Amplía la sala y reduce el tamaño del comedor».
- «Reduce el presupuesto con materiales económicos».

Cada instrucción crea una versión nueva e indica qué cambió. Si el cambio no es viable, el sistema lo explica en lugar de romper el diseño.

16.5 Comparar, deshacer y exportar

- «Comparar» muestra áreas, costos y el listado de cambios entre dos versiones.
- «Deshacer» y «Rehacer» restauran versiones como versiones nuevas, sin perder historial.
- «Plano SVG» descarga la planta del nivel activo; «Ficha PDF» genera el informe del proyecto.
- «VR» y «AR» inician una sesión inmersiva si el navegador y el dispositivo lo permiten; si no, se informa y se mantiene el modo 3D.

16.6 Revisión profesional

Un usuario con rol de arquitecto o administrador puede registrar la revisión de una versión (`POST /projects/:id/approvals`), lo que cambia el estado del proyecto. Los documentos generados recuerdan en su pie que son preliminares y requieren validación profesional.

17. Datasets públicos recomendados y licencias

Fuente	Aporte	Uso previsto en el producto	Licencia / advertencia
HouseExpo	35 126 planos 2D y 252 550 habitaciones	Patrones de distribución y evaluación de propuestas	Verificar términos del repositorio antes de uso comercial
HouseExpo++	Grafo de conectividad y semántica de ambientes	Validación de accesibilidad y privacidad	Derivado de HouseExpo; revisar términos
3D-FRONT	6 813 viviendas CAD, 18 968 habitaciones y 13 151 muebles	Mobiliario, texturas y escenas interiores	Uso académico; licencia específica por componente
Structured3D	Escenas interiores con anotaciones estructurales	Comprensión de muros, vanos y superficies	Requiere acuerdo de acceso; no asumir uso comercial
buildingSMART IFC	Modelos BIM de referencia en IFC 4.x	Interoperabilidad futura y pruebas de importación	Modelos de ejemplo con distintos niveles de validación
Dataset propio de instrucciones	JSONL de instrucción → operación → validación	Evaluación del intérprete y del motor	Generado en este proyecto; formato documentado en el encargo
Catálogo propio de materiales	Precios, proveedor y disponibilidad	Personalización y presupuesto	Datos de demostración; sustituir por datos reales

Ningún dataset público se incorporó al repositorio: el MVP funciona con datos de demostración propios. La integración de estas fuentes exige revisar licencias, atribución y restricciones de redistribución, en particular para texturas y objetos 3D.

18. Glosario

Término	Definición
Ambiente	Espacio delimitado de la vivienda (sala, cocina, dormitorio) representado por un rectángulo con superficie y altura.
Muro	Segmento con espesor, altura y tipo (exterior, tabique o baranda) que delimita ambientes.
Vano	Abertura en un muro: puerta o ventana, con posición, ancho, altura y antepecho.
Huella	Proyección en planta de la vivienda dentro del terreno.
Retiro	Distancia mínima obligatoria entre la edificación y el límite del terreno.
Snapshot	Copia inmutable del modelo completo asociada a una versión.
Operación	Unidad atómica de cambio sobre el modelo (ampliar, agregar, cambiar material).

Término	Definición
Validación	Conjunto de reglas geométricas y funcionales que una operación debe cumplir.
Diff	Diferencia estructurada entre dos versiones del modelo.
WebXR	API del navegador para sesiones de realidad virtual y aumentada.
BIM / IFC	Modelado de información de construcción y su formato abierto de intercambio.
PGlite	Distribución de PostgreSQL compilada a WebAssembly, ejecutable en proceso.

Este documento describe el estado real del prototipo. Cualquier ampliación debe mantener la separación entre interpretación, geometría y presentación, y registrar en el historial cada cambio que afecte el diseño.

Fin del documento.